

STACK

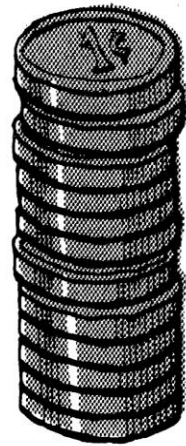
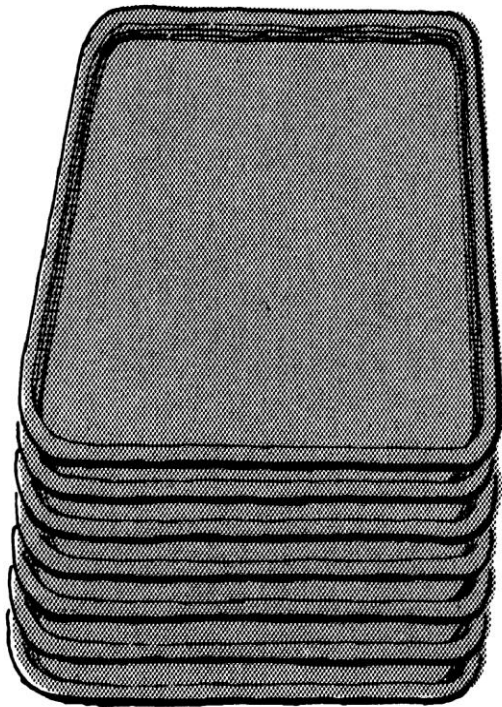
CHAPTER 03

Stack

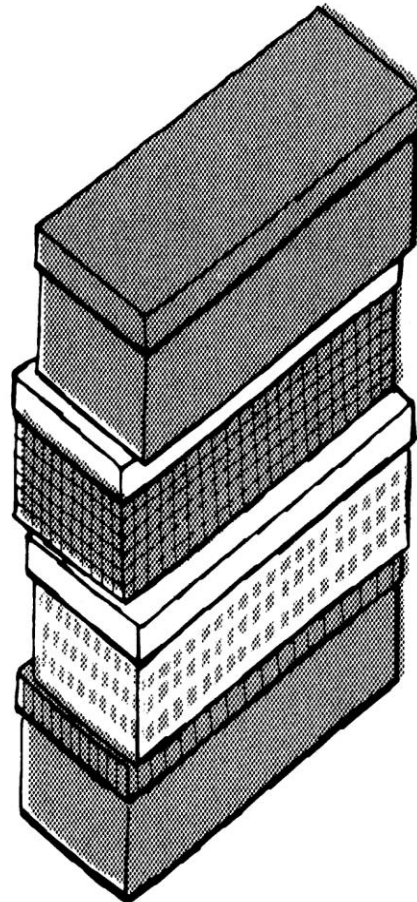
- Stacks are linear lists.
- All deletions and insertions occur at one end of the stack known as the TOP.
- Data going into the stack first, leaves out last.
- Stacks are also known as LIFO data structures (**Last-In, First-Out**).

Stack

A stack of
cafeteria trays

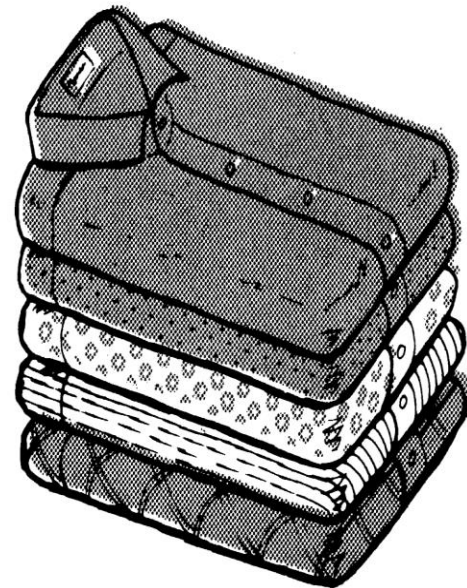


A stack
of pennies



A stack of shoe boxes

A stack of
neatly folded shirts

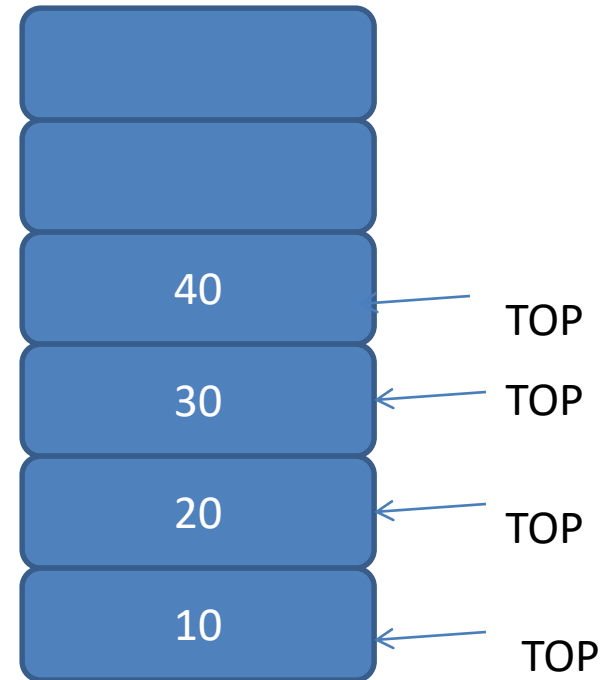


OPERATIONS ON THE STACK

- push – Adds an item to the top of a stack.
- pop – Removes an item from the top of the stack and returns it to the user.

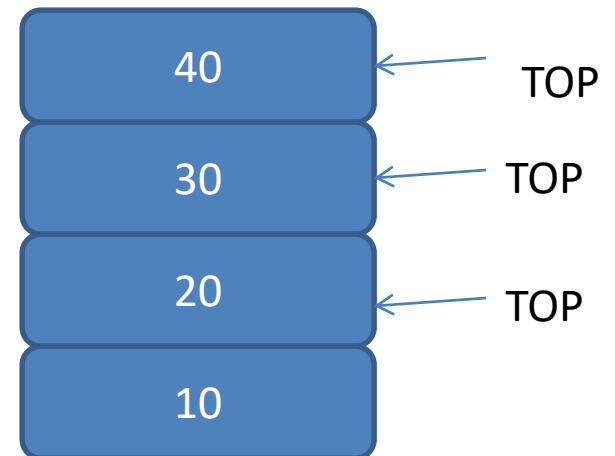
STACK

- DATA :- 10,20,30,40,50,60
- PUSH 10
- PUSH 20
- PUSH 30
- PUSH 40
- PUSH 50
-



STACK OPERATIONS

- POPPED ELEMENTS
- 40
- 30
- 20



APPLICATIONS OF STACK

- INFIX TO POSTFIX CONVERSION
- INFIX TO PREFIX CONVERSION
- STRING REVERSE
- EVALUATION OF POSTFIX EXPRESSION
- CHECK WHETHER THE EXPRESSION IS VALID OR NOT

INFIX TO POSTFIX CONVERSION

- EXPRESSIONS
- Prefix: $+ a b$
- Infix: $a + b$
- Postfix: $a b +$

Infix to Postfix Conversion

- There are two methods
 - Manual method(Parenthesis)
 - stack method

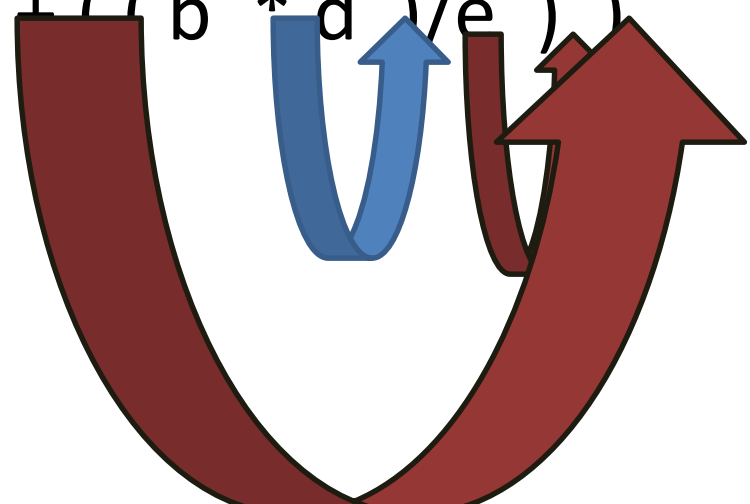
Manual Method(Infix to Postfix)

- $A + b * d / e$

- $A + (b * d) / e$

- $A + ((b * d) / e)$

- $(A + ((b * d) / e))$

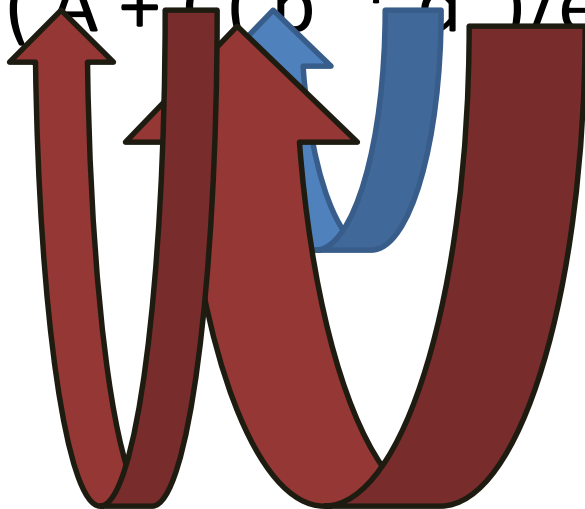


$ABD * E / +$

Manual Method(Infix to Prefix)

- $A + b * d / e$
- $A + (b * d) / e$
- $A + ((b * d) / e)$
- $(A + ((b * d) / e))$

$+A/*BDE$



Infix to Postfix using stack

$$A + B * C - D / E$$

<u>Infix</u>		<u>Stack (bot->top)</u>	<u>Postfix</u>
a) A + B * C - D / E			
b) + B * C - D / E			A
c) B * C - D / E	+		A
d) * C - D / E	+		A B
e) C - D / E	+	*	A B
f) - D / E	+	*	A B C
g) D / E	+	-	A B C *
h) / E	+	-	A B C * D
i) E	+	- /	A B C * D
j)	+	- /	A B C * D E
k)			A B C * D E / - +

Infix to Prefix using stack

- Reverse the infix string .
- Replace '(' with ')' and ')' with '('.
- Convert it to postfix.
- Reverse the result.

Postfix evaluation

Operand: push

Operator: pop 2 operands, do the math, pop result
back onto stack

1 2 3 + *

Postfix

a) 1 2 3 + *
b) 2 3 + *
c) 3 + *
d) + *
e) *
f)

Stack(bot -> top)

1
1 2
1 2 3
1 5 // 5 from 2 + 3
5 // 5 from 1 * 5